

SDK / стек

Параметр	Значение
Название приложения	Tigrano Maze Zip Cat
Android Gradle Plugin	8.11.1
minSdk	24
targetSdk	36
Kotlin	да, 2.2.0
Web View	да
Custom Tabs	нет
Рекламные сети	нет
Аналитика	Firebase Cloud Messaging, Firebase Installations, Firebase Realtime Database
Permissions	android.permission.INTERNET, android.permission.ACCESS_NETWORK_STATE, android.permission.POST_NOTIFICATIONS, android.permission.WAKE_LOCK, com.google.android.c2dm.permission.RECEIVE, com.devanos.nilufar.usmonova.DYNAMIC_RECEIVER_NOT_EXPORTED_PERMISSION
Libraries	Flutter (io.flutter.embedding v2), Kotlin 2.2.0, Jetpack Compose (foundation/material3/ui 1.7.5), AndroidX AppCompat 1.7.0, Activity 1.9.3, Core 1.13.1, Lifecycle 2.8.7, DataStore Preferences 1.1.1, Emoji2, ProfileInstaller, Startup; Retrofit2, OkHttp3, Moshi (Square); Firebase Database, Firebase Messaging, Firebase Installations, Firebase Common, Play Services (base/basement/tasks/cloud-messaging); kotlinox-coroutines
Подозрительные домены	date123456.com, ranoaudio-mix-default-rtdb.europe-west1.firebaseio.com
SharedPreferences	DataStore Preferences «p7_prefs»: ключи install_id, idfv, p7_pd (принятие policy), fcm_token, pending_click_id (click_id из deep link до отправки на сервер)
Есть ли клоака	да
Подозрительные слова	click_id, pending_click_id, keitaro_off, open_policy, block_token, antiFraudRepository, Gate, S0, MainStub, OpenPolicy, hApi, hFp, save_link, startup, persist, WebView, webview url, Decision, RTDB

Проверка домена: date123456.com

Параметр	Значение
Домен	date123456.com
VirusTotal	нет (домен не входил в готовый domain_checks пайплайна)
Куда редиректит	нет (DNS не резолвится: Name or service not known)
Цепочка редиректов	нет
Что выводит (кратко)	Хост из XOR-строк приложения: белый список privacy-хоста. Сейчас домен не отвечает (NXDOMAIN/нет DNS).
Где припаркован	нет

Проверка домена: ranoaudio-mix-default-rtdb.europe-west1.firebaseio.com

Параметр	Значение
Домен	ranoaudio-mix-default-rtdb.europe-west1.firebaseio.com
VirusTotal	ошибка: домен не найден в VirusTotal
Куда редиректит	нет (без редиректа)
Цепочка редиректов	https://ranoaudio-mix-default-rtdb.europe-west1.firebaseio.com/
Что выводит (кратко)	Firebase error. Please ensure that you have the URL of your Firebase Realtime Database instance configured correctly.

Как работает клоака

1) Короткий итог

Снаружи это выглядит как простая мобильная игра/лабиринт: на устройстве подписано «Tigrano Maze Zip Cat», внутри есть обычный Flutter-экран (MainActivity). Но при запуске пользователь сначала попадает не в игру, а в «шлюз» GateActivity — тихий экран решения.

Обман в том, что модератору Google и «не тому» трафику показывают безопасный режим: локальную Privacy Policy или саму игру-заглушку. «Нужному» пользователю сервер присылает другую ссылку — её открывают во встроенном браузере (WebView, S0FrameActivity), и человек оказывается на внешнем лендинге/оффере.

Финал для боевого трафика — динамический URL из ответа startup-API (поле n6u4xl). В APK конечный казино/оффер домен не зашит: его отдаёт удалённый сервер, адрес которого приложение сначала читает из Firebase Realtime Database. В коде явно фигурирует флаг keitaro_off — типичный признак связки с трекером Keitaro.

2) Кому что показывают

Белый / обычный режим: модератор, бот, «заблокированный» или «нецелевой» пользователь по решению сервера. Что видит: Lc0Activity с локальной Privacy Policy из assets (file:///android_asset/p9/index.html) — нейтральный текст про privacy; либо Flutter MainActivity («MainStub») — обычная игра, когда ответ говорит ok + keitaro_off. При ошибках/таймауте gate тоже уводят в MainActivity (fallback).

Чёрный / боевой режим: пользователь, которому startup-ответ вернул непустой n6u4xl (URL назначения). Что открывается: S0FrameActivity — полноэкранный WebView с JavaScript и DOM Storage, загрузка этой ссылки. Это уже не геймплей лабиринта, а веб-контент по удалённому URL.

3) Как приложение решает, кого куда пустить

Firebase Realtime Database (удалённый конфиг). Что смотрит: узел Rt0 на ranoaudio-mix-default-rtdb... Условие: за 5 секунд пришли поля hApi (базовый URL API) и опционально hFp. Результат: baseUrl для Retrofit записывается в RuntimeConfig; без hApi startup не к чему ходить.

Startup API + устройство/гео. Что смотрит клиент отправляет: locale (язык-страна), country, model, manufacturer, размер экрана, installId, idfv, appVersion — на endpoint {ep}=startup с обфусцированными query-именами (b8q2la, c5v7nk, ...). Условие и фильтрация на сервере (в APK серверной логики нет). Результат: JSON StartupResponseDto с полями-псевдонимами.

Разбор ответа (Decision на клиенте). 1) Если n6u4xl не пустой → OpenS0 (боевой WebView). 2) Иначе если a2c8yo == open_policy → OpenPolicy (privacy). 3) Иначе если v9m2pt не пустой → fallback block_token → уводят в игру. 4) Иначе если z3q8rs == ok и l7s5hk == keitaro_off → OpenMainStub (белая Flutter-игра). 5) Иначе если ok без URL → снова Policy. 6) Иначе fallback по e4f7nr / unknown_response → MainActivity.

Deep link click_id. Что смотрит: query-параметр click_id у http/https intent. Условие: параметр есть. Результат: кладётся в pending_click_id и позже POST'ится на сервер (атрибуция клика/оффера).

Таймаут шлюза 45 с. Что смотрит: успел ли прийти decision. Условие: timeout → gate_timeout. Результат: показывают MainActivity (безопасный fallback).

Отдельного явного списка ботов/VPN на клиенте в коде не видно — отбор делает сервер (и, судя по keitaro_off, скорее всего Keitaro/TDS). Клиент лишь честно передаёт страну/локаль/модель.

4) Цепочка по шагам

Шаг 1. Пользователь запускает приложение → стартует GateActivity (LAUNCHER) → зачем: не сразу отдать игру, а сначала решить режим → на экране пустой «шлюз» Compose.

Шаг 2. Если пришли по ссылке с click_id → extractAndStoreClickId пишет pending_click_id в DataStore → зачем: сохранить идентификатор рекламного клика → система готовит postClick на API.

Шаг 3. Клиент читает Firebase RTDB .../Rt0 (hApi, hFp) → зачем: узнать актуальный адрес «мозга» без хардкода в сторе → RuntimeConfig.baseUrl обновлён.

Шаг 4. antiFraudRepository делает GET startup с locale/country/model/installId/... → зачем: сервер оценит трафик → клиент получает StartupResponseDto.

Шаг 5. Decision разбирает поля: при наличии п6u4xl выбирает OpenS0 → зачем: открыть боевой URL → uiState = S0(dest=URL).

Шаг 6. GateActivity стартует S0FrameActivity, передаёт URL в extra x0d → WebView грузит ссылку с JS → пользователь видит лендинг/оффер.

Шаг 7. На onPageFinished WebView «запоминает» чужой http(s)-хост (не firebase / не date123456.com) через persist/save_link → зачем: отправить финальный URL на сервер → атрибуция/логирование перехода.

Шаг 8. Если сервер сказал open_policy / ok без URL → Lc0Activity показывает локальную Privacy Policy → белая витрина для сторa.

Шаг 9. Если ok + keitaro_off или ошибка/block_token/timeout → MainActivity (Flutter-игра) → модератор/отклонённый трафик видит «настоящее» приложение.

Шаг 10. Параллельно FCM-токен пишется в prefs и уходит postPush → зачем: пуши/ретаргет → сервер знает устройство.

5) Технические уловки

XOR-шифрование строк (ключ в AbstractC1473m3.k). URL Firebase, https, open_policy, keitaro_off, save_link, privacy asset path, host date123456.com спрятаны за XOR. Зачем: усложнить grep по APK. В итоге расшифровываются рабочие URL и команды решения.

Обфусцированные имена полей API (z3q8rs, п6u4xl, a2c8yo, l7s5hk, query b8q2la...). Зачем: скрыть смысл «status / dest URL / action / keitaro flag» от поверхностного анализа.

Динамический baseUrl из RTDB (hApi). В манифесте/строках нет боевого API-хоста оффера — только Firebase. Зачем: менять сервер клоаки без обновления APK в Google Play.

WebView без Custom Tabs. Боевой контент держат внутри приложения (S0Frame), внешние схемы — через ACTION_VIEW. Зачем: контроль навигации, перехват privacy-URL, persist финального хоста.

Локальная белая Privacy Policy в assets/p9. Не зависит от сети. Зачем: всегда есть «чистая» страница для open_policy.

6) Роль подозрительных доменов

ranoaudio-mix-default-rtddb.europe-west1.firebaseio.com — «мозг» конфигурации на шагах 3–4. С него читается Rt0/hApi (базовый URL API решения). Сейчас корень отдаёт ошибку Firebase (инстанс не сконфигурирован или закрыт снаружи); для живого приложения нужен валидный путь/auth. Это не лендинг оффера, а источник адреса сервера клоаки.

date123456.com — зашитый «белый» хост privacy. На шагах WebView: если URL ведёт на этот хост (https) или на package/privacypolicy, приложение открывает read-only Policy и не считает страницу боевым оффером для save_link. Сейчас DNS не резолвится — похоже на заготовку/мертвый белый домен. Не «мозг» решения и не финальный оффер.

7) Финал для пользователя

После всех проверок «нужный» пользователь оказывается на веб-странице по URL из поля п6u4xl ответа startup (тип сайта в статическом коде не зафиксирован: казино/беттинг/оффер — динамически с сервера, с намёком на Keitaro через keitaro_off). «Ненужный» видит либо локальную Privacy Policy, либо Flutter-игру Tigrano Maze Zip Cat. Итоговый оффер URL без успешного ответа живого hApi+startup из приложения сейчас не извлекается — он приезжает только с API.